COMP 3311
DATABASE MANAGEMENT SYSTEMS

LECTURE 12
NOSQL
DATABASE MANAGEMENT SYSTEMS

NOSQL DATABASE MANAGEMENT SYSTEMS: OUTLINE

Motivation

Types of NoSQL DBMSs

JavaScript Object Notation (JSON)

MOTIVATION

- Very large volumes of data (big data) are being collected continuously.
  - Driven by growth of the web, social media, point-of-sale transactions, sensors and, more recently, the internet-of-things.

- **Big data** is differentiated from data handled by earlier generation database systems.

  Volume much larger amounts of data stored – and continuously growing.

  Velocity much higher rates of data generation (insertions) and data analysis.

  Variety many types of data formats (unstructured, semi-structured), beyond relational (structured) data.

- Many applications are willing to sacrifice relational DBMS features if they can get very high scalability.

THE NOSQL MOVEMENT

- Relational database systems emphasize keeping data consistent.
  - formal schema, data types, referential integrity, transactions, etc.
- The focus on consistency may hamper flexibility and scalability.
  - Vertical scaling: extend server storage capacity and/or CPU power.
  - Horizontal scaling: add more servers connected in a network.
- Relational database systems are *not good* at extensive horizontal scaling due to large coordination overhead because of the focus on consistency.
- Rigid database schemas and rich querying functionality are often overkill for many applications resulting in a need for DBMSs that can deal effectively with
  - massive data volumes, flexible data structures, scalability and availability

NoSQL DBMSs: Key Ideas

- Store and manipulate data in formats other than relations (i.e., non-relational databases).
- Aim at near-linear horizontal scalability with high availability.
  - Rather than placing data on a central server, it is distributed over several servers to improve performance as well as availability.
- Introduce the concept of eventual consistency (versus always consistent).
  - Data, and its replicas, will *eventually become consistent at some point in time* after each transaction, but *continuous and instantaneous consistency is not guaranteed*.
- Provide much simpler querying functionality and/or application programming interfaces (APIs).

TYPES OF NOSQL DBMSS

- key-value

  Examples: BerkeleyDB, LevelDB, Memcached, Redis, Riak

- document

  Examples: CouchDB, MongoDB, OrientDB, RavenDB, Terrastore

- columnar

  Examples: Amazon SimpleDB, Cassandra, HBase, Hypertable

- graph

  Examples: FlockDB, HyperGraphDB, Neo4J, OrientDB

KEY-VALUE DBMSS

- A key-value DBMS stores large numbers (billions or even more) of small (KB-MB) sized records.

- Data is stored as key-value pairs.
  - The keys are unique and are the sole search criterion to retrieve the corresponding value.
  - The values are usually uninterpreted bytes that contain no schema information.

- Basic data manipulation operations:

put(key, value)	store the value using the specified key.
get(key)	retrieve the value associated with the specified key.
delete(key)	remove the specified key and its associated value.

DOCUMENT DBMSS

- A document DBMS is a kind of key-value DBMS where the values are a collection of unordered key-value pairs, representing semi-structured data (like XML data).

- The value of the key can be user-defined or auto-generated.

```
{
      "_id": ObjectId("56349fh4ht49jv4j9jv4jvj94jv49"),
      "title": "Harry Potter"
      "isbn": "111-1111111111"
      "authors": [ "J.K. Rowling" ]
      "price:" 32
      "dimensions:" "8.5 x 11.0 x 0.5"
      "pageCount": 234
      "genre": "Fantasy"
}
```

- Besides the basic key-value operations, queries on non-key attributes are also supported.

- Documents are often represented using **JSON** (JavaScript Object Notation), **BSON** (Binary JSON), **YAML** (YAML Ain't Markup Language) or **XML**.

☞ **JSON is most widely used in NoSQL DBMSs.**

COLUMN-ORIENTED DBMSS

Id	Genre	Title	Price	Audiobook price
1	fantasy	My first book	20	30
2	education	Beginner's guide	10	null
3	education	SQL strikes back	40	null
4	fantasy	The rise of SQL	10	null

Query: Find books with Price=10.

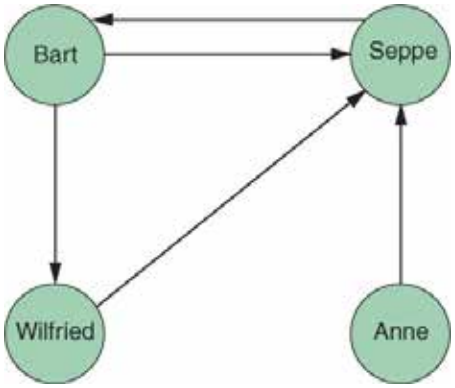
- A row-oriented DBMS needs indexes to efficiently answer this query, which adds overhead when updates are frequent.

- A column-oriented DBMS can directly process this query.
  - Null values do not take up storage space (e.g., **Audiobook price** has only one book id).

Genre:	fantasy: 1,4	education: 2,3		
Title:	My first book: 1	Beginner's guide: 2	SQL strikes back: 3	The rise of SQL: 4
Price:	20: 1	10: 2,4	40: 3	
Audiobook price:	30: 1			

GRAPH DBMSS

- A graph DBMS stores information as nodes and edges.



- Relationship tables are replaced by more interesting and semantically meaningful relationships that can be navigated and/or queried using graph-traversal based on graph-pattern matching.

- One-to-one, one-to-many, and many-to-many relationships can easily be modeled in a graph.

GRAPH DBMSS (CONT'D)

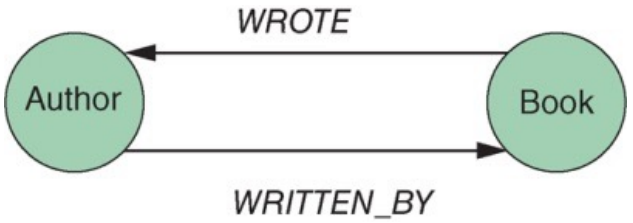
- For an N:M relationship between books and authors, a relational DBMS needs three tables: Book, Author and BookAuthor to represent the relationship.

- An SQL query to return all book titles for books written by a particular author would look like:

```
select title
from Book, Author, BookAuthor
where Author.id=BookAuthor.authorId
   and Book.id=BookAuthor.bookId
   and Author.name='Bart Baesens';
```

- In a graph DBMS, the schema would be the graph below right, and the query would look like (using the Cypher query language):

```
match (b:Book)←[:WRITTEN_BY]-(a:Author)
where a.name = "Bart Baesens"
return b.title
```



JAVASCRIPT OBJECT NOTATION (JSON)

- **JavaScript Object Notation** (**JSON**) is a standardized, lightweight, text-based notation used to define objects and arrays for data-interchange.
  - ☞ Used to serialize and transfer structured data over the Internet.
- Self-describing — easy to understand, manipulate and generate; programming language independent.
- Based on a subset of the **JavaScript** programming language.
- Supported by all major **JavaScript** frameworks and most backend technologies.
- Used extensively in document-based NoSQL DBMSs for storing data.

JSON SYNTAX

```
{
   "id": "22222",
   "name":  {
                "firstName": "Albert",
                "lastName": "Einstein"
             },
   "deptName": "Physics",
   "children":[
                 { "firstName": "Hans", "lastName": "Einstein" },
                 { "firstName": "Eduard", "lastName": "Einstein" }
               ]
};
```

JSON example viewed in a browser

Einstein Record

Id = 2222

Name = Albert Einstein

Dept = Physics

Children = Hans Einstein, Eduard Einstein

JSON SYNTAX

- An object definition is enclosed in curly braces ({ }) and is composed of an unordered collection of key-value pairs.
  - ➢ Objects can be nested.
  - – A key is a string that must be enclosed in double quotes (" ") and is followed by a colon (:).
  - – A value can be an instance of an object, array, number, string, Boolean (true, false), or null.
    - ➢ Key-value pairs are separated by a comma (,).
    - ➢ Strings must be enclosed in double quotes (" ").
- An array definition is enclosed in square brackets ([ ]) and is composed of an unordered collection of values.
  - ➢ Array elements are separated by a comma (,).

JSON DATA TYPES AND VALUES

object	an unordered collection of key-value pairs, separated by commas, enclosed in braces { }.
array	an unordered collection of values, separated by commas, enclosed in brackets [ ].
string	a sequence of zero or more characters enclosed in double quotes. A \ (backslash) is the escape character (e.g., to allow a double-quote or other control character to be part of the string).
number	integer, real, scientific notation (can be fractional; cannot contain superfluous zeros). Must not be enclosed in double quotes.
Boolean	can only take on the values true or false.

ORACLE SQL/JSON FUNCTIONS

json_object / json_array

These **SQL** functions construct either a collection of **JSON** objects or arrays, respectively, from the tuples of a **SQL** query result.

json_objectagg / json_arrayagg

These aggregate **SQL** functions construct either a single **JSON** object or array, respectively, from the tuples of a **SQL** query result.

- These functions can be nested in a **select** statement to generate complex, hierarchical **JSON** documents.
- **JSON** values within the returned document are derived from **SQL** values in the input as follows:
  - A **SQL** number is converted to a **JSON** number.
  - A non-null, non-number **SQL** value is converted to a **JSON** string.
  - A **SQL** null value is handled by an optional null-handling clause.

ORACLE SQL/JSON FUNCTIONS (CONT'D)

Optional clauses

null-handling clause — determines how a **SQL** null value resulting from input evaluation is handled.

- **null on null** — An input **SQL** null value is converted to **JSON** null value for output. This is the default behavior for **json_object** and **json_objectagg**.
- **absent on null** — An input **SQL** null value results in no corresponding value output. This is the default behavior for **json_array** and **json_arrayagg**.

returning clause — the **SQL** data type used for the function return value. The default is varchar2(4000).

- Needed if an output document will be more than 4,000 characters.

strict keyword — checks the returned **JSON** data to be sure it is well-formed. If **strict** is present and the returned data is not well-formed then an error is raised.

with unique keys keywords - ensures there are no duplicate **JSON** keys.

SQL/JSON FUNCTION: JSON_OBJECT

json_object('key' value expression, … [null-handling clause])

- The json_object function constructs a collection of JSON objects (one or more) from key–value pairs that are provided as arguments.
  - Each key is enclosed in single-quotes and must evaluate to a SQL string.
  - Each expression can be any SQL expression (but is usually an attribute value).
  - For each tuple in the query result, a JSON object that contains an object member for each key-value pair is returned.
- Object member order is undetermined but can be set with an order by clause of the enclosing select statement.

☞ **The default null-handling behavior for json_object is null on null.**

SQL/JSON FUNCTION: JSON_OBJECT (CONT'D)

Example: Construct a JSON object for each tuple in the Client relation with fields client id, name and district; a field whose value is null should be absent.
Order the collection of objects by district ascending.

```
select json_object(
    'clientId'   value clientId,
    'name'       value name,
    district'    value district
    absent on null)
from Client
order by district;
```

Query result:

```
{"clientId": 13, "name": "Steven Chan",
    "district": "Central and Western"}
{"clientId": 4, "name": "Mary Kwan",
    "district": "Islands"}
{"clientId": 8, "name": "Isaac Li"}
{"clientId": 17, "name": "Pam Du"}
```

☞ **One object is created for each tuple in the query result.**

json_object null-handling
The default behaviour for json_object is null on null.

SQL/JSON FUNCTION: JSON_ARRAY

json_array(expression, … [null-handling clause])

- The json_array function constructs a collection of JSON arrays from the results of evaluating its argument expressions.
  - Each expression can be any SQL expression which must evaluate to a JSON string, number or null value.
  - Each evaluated expression is an explicit array element value (except when an expression evaluates to SQL null and the absent on null clause is included).
  - For each tuple in the query result, each expression is converted to a JSON value, and a JSON array is returned that contains those JSON values.
  - Array element order is the same as the expression order.
    - If expression is a select statement, then the elements returned by the select statement can be ordered.

☞ **The default null-handling behavior for json_array is absent on null.**

SQL/JSON FUNCTION: JSON_ARRAY (CONT'D)

Example: Construct a JSON array for each tuple in the Client relation with elements the values of client id, name and district; an array element whose value is null should be included.
Order the array elements by district ascending.

```
select json_array(
    clientId,
    name,
    district
    null on null)
from Client
order by district desc;
```

Query result:

[3, "Steven Chan", "Central and Western"]
[4, "Mary Kwan", "Islands"]
[8, "Isaac Li", null]
[17, "Pam Du", null]

☞ **One array is created for each tuple in the query result.**

json_array null-handling
The default behaviour for **json_array** is **absent on null**.

SQL/JSON FUNCTION: JSON_OBJECTAGG

json_objectagg('key' value expression [null-handling clause])

- The json_objectagg function constructs a single **JSON** object by aggregating key-value pairs from multiple rows of a **SQL** query as the object members.

Example: Construct a single JSON object for the Client relation with fields district, one for each Client; a district field whose value is null should be absent.

```
select json_objectagg(
    'district' value district
    absent on null)
from Client;
```

json_objectagg null-handling
The default behaviour for **json_objectagg** is **null on null**.

Query result: {"district": "Central and Western", "district": "Islands"}

☞ **A single object is created from all the tuples in the query result.**

SQL/JSON FUNCTION: JSON_ARRAYAGG

json_arrayagg(expression [order by clause] [null-handling clause])

- json_arrayagg constructs a single JSON array by aggregating the values of the expression argument from multiple rows of a SQL query as the array elements.

Example: Construct a single JSON array for the Client relation with elements the values of all districts; an array element whose value is null should be included. Order the array elements by district descending.

```
select json_arrayagg(
    district order by district desc
    null on null)
from Client;
```

json_arrayagg **null-handling**
The default behaviour for json_arrayagg is **absent on null**.

Query result: [null, null, "Islands", "Central and Western"]

☞ **A single array is created from all the tuples in the query result.**

NOSQL DBMSS: SUMMARY

	Relational DBMSs	NoSQL DBMSs
Data organization paradigm	Relational tables	Key-value based Document based Column based Graph based Others: XML, object based, time series, probabilistic, etc.
Distribution	Usually single server; can be distributed	Usually multiple distributed servers
Scalability	Vertical scaling; harder to scale horizontally	Easy horizontal scaling, easy data replication
Openness	Closed and open source	Mainly open source
Schema role	Schema-driven	Mainly schemaless or flexible schema
Query language	**SQL**	No or simple querying facilities, or special-purpose languages
Feature set	Many features (triggers, views, stored procedures, etc.)	Simple API
Data volume	Capable of handling normal-sized datasets	Capable of handling huge amounts of data and/or very high frequencies of read/write requests